Paulina Gómez García

Pablo Andres Carrillo Yepes

1. Análisis del problema y consideraciones de la solución

1.1 Descripción del problema

El proyecto Ude@Book consiste en el diseño e implementación de la capa lógica de una red social, desarrollada en C++ bajo la programación orientada a objetos, utilizando memoria dinámica y estructuras de datos de creación propia. El objetivo central es gestionar eficientemente la información de una comunidad virtual de usuarios y sus relaciones, optimizando tanto el consumo de memoria como la cantidad de iteraciones requeridas por cada funcionalidad.

El sistema debe permitir el inicio de sesión, la visualización aleatoria ponderada de fotos de perfil de amigos, el envío y aceptación de solicitudes de amistad, la visita de perfiles y álbumes de amigos en modo carrusel, la generación de sugerencias de amistad según amigos en común y la administración de mensajes. Además, cada funcionalidad debe reportar las iteraciones realizadas y la memoria consumida. La interacción se realiza por consola, y la información se carga desde archivos y se actualiza en ellos.

1.2 Alcance y restricciones

La solución no utiliza contenedores de la STL, structs, herencia ni librerías externas que resuelvan aspectos esenciales. Quedan fuera del alcance el registro y la eliminación de cuentas, la creación o edición de álbumes y fotos, la eliminación de amistades y la respuesta directa a mensajes.

1.3 Identificación de las clases

Las clases del modelo se identificaron a partir de las entidades descritas en el enunciado, cada una con información y comportamiento propios:

- **Usuario:** entidad central del sistema.
- **Álbum y Foto:** representan el contenido multimedia, con su configuración de privacidad y su conteo de likes.
- **Mensaje:** modela la comunicación entre usuarios y su estado de lectura.
- **Fecha:** se reutiliza en la fecha de nacimiento, de inscripción, de captura de fotos y de envío de mensajes, lo que facilita las comparaciones y las búsquedas por rango.
- **Sugerencias:** genera las sugerencias de amistad a partir de los amigos en común.

1.4 Gestión del conjunto de usuarios

El conjunto de todos los usuarios de la plataforma se almacena mediante miembros estáticos de la clase Usuario: un arreglo dinámico compartido por todas las instancias, la cantidad de usuarios registrados y la matriz de amistades. Las relaciones reflexivas de Usuario en el diagrama (amistades y solicitudes) representan relaciones entre instancias de la misma clase; en particular, la relación de amistad se implementa mediante la matriz de adyacencia descrita en la sección 1.5.

El índice de cada usuario corresponde a su posición en el arreglo y se usa como su fila y columna en la matriz. Para que la búsqueda por nombre sea eficiente, la carga sigue este orden:

1. Se leen los usuarios desde el archivo y se almacenan en el arreglo dinámico.
2. Se ordena el arreglo por nombre de usuario mediante Merge Sort, con complejidad O (n log n).
3. Se asigna a cada usuario su índice según su posición en el arreglo ordenado.
4. Se leen las amistades, guardadas como parejas de nombres de usuario; cada nombre se ubica con búsqueda binaria y se activan las posiciones correspondientes de la matriz.

Con el arreglo ordenado, la búsqueda de un usuario por su nombre se realiza en O (log n), unas 10 iteraciones para 1000 usuarios, frente a las hasta 1000 de una búsqueda lineal. Guardar las amistades por nombre y no por índice hace que el archivo no dependa del orden interno de los usuarios.

1.5 Representación de las amistades

Las relaciones de amistad se representan mediante una matriz de adyacencia completa de N×N bits, donde el bit [i][j] vale 1 si los usuarios con índices i y j son amigos. Cada fila se almacena como un arreglo dinámico de bytes, por lo que para 1000 usuarios la matriz ocupa aproximadamente 125 KB, ocho veces menos que una matriz de valores booleanos. Esta matriz es única y compartida por todo el programa, y la utilizan las funcionalidades de solicitudes, mensajes, visita de perfiles, fotos de perfil de amigos y sugerencias.

Como la amistad es mutua, la matriz es simétrica y su diagonal es cero. Esta propiedad se aprovecha al almacenar las amistades en el archivo, donde solo se guardan las parejas de la diagonal superior (i < j) para no duplicar información; al cargarlas o al aceptar una solicitud, se activan ambas posiciones, [i][j] y [j][i].

Se optó por la matriz completa en lugar de almacenar solo la mitad triangular porque mantiene contiguos todos los amigos de un usuario en una sola fila. Esto permite calcular los amigos en común entre dos usuarios aplicando una operación AND byte a byte entre sus filas y contando los bits activos, lo que reduce el costo de cada comparación de N a N/8 operaciones. Para generar las sugerencias se compara la fila del usuario con la de los demás, se excluyen él mismo y sus amigos actuales, y los candidatos se ordenan según la cantidad de amigos en común.

3. Formato de los archivos

La información se almacena en 5 archivos de texto plano, uno por tipo de entidad, con una línea por registro y los campos separados por el carácter |. Se eligió este separador porque no aparece en nombres de usuario, correos, ciudades ni fechas. Las fechas se escriben en formato DD/MM/AAAA.

ARCHIVO	UNA LINEA POR	CAMPOS
usuarios.txt	usuario	usuario | contraseña | correo | fechaNacimiento | ciudad | pais | fechaInscripcion
amistades.txt	amistad	usuarioA | usuarioB
solicitudes.txt	solicitud pendiente	emisor | receptor
mensajes.txt	mensaje	emisor | receptor | fecha | leido | texto
albumes.txt	álbum o foto	A | usuario | nombreAlbum | privacidad, seguida de F |

ARCHIVO	UNA LINEA POR	CAMPOS
		nombreFoto | fechaCaptura | tamaño | likes

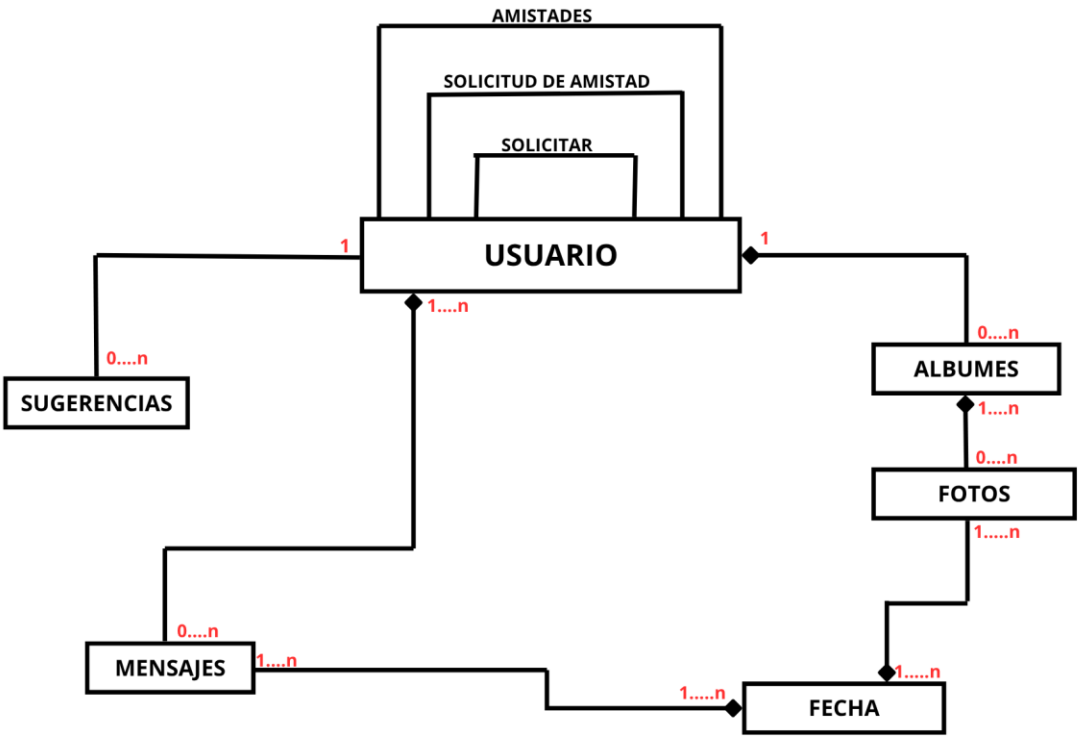
3.1 Consideraciones del formato

- **Amistades:** cada pareja se escribe una sola vez, correspondiente a la diagonal superior de la matriz. Se guardan nombres de usuario y no índices, para que el archivo no dependa del orden interno del arreglo.
- **Mensajes:** el campo leido vale 0 (no leído) o 1 (leído). El texto va como último campo, de modo que se lee todo el resto de la línea y un carácter | escrito por el usuario no rompe el registro. Los mensajes de un mismo día conservan el orden en que se registraron en el archivo.
- **Álbumes y fotos:** cada línea que empieza con A define un álbum, y las líneas F que la siguen son sus fotos. La privacidad se representa con un número: 0 = público, 1 = privado, 2 = amigos.
- **Rutas:** no se almacenan. Como todas siguen el patrón /users/(sub-ruta-X-fija)/usuario/albums/álbum/foto.png, se construyen a partir de los nombres al momento de mostrar la foto, lo que reduce el espacio en archivo y en memoria. La foto de perfil se construye igual, con el nombre fijo profile.png.

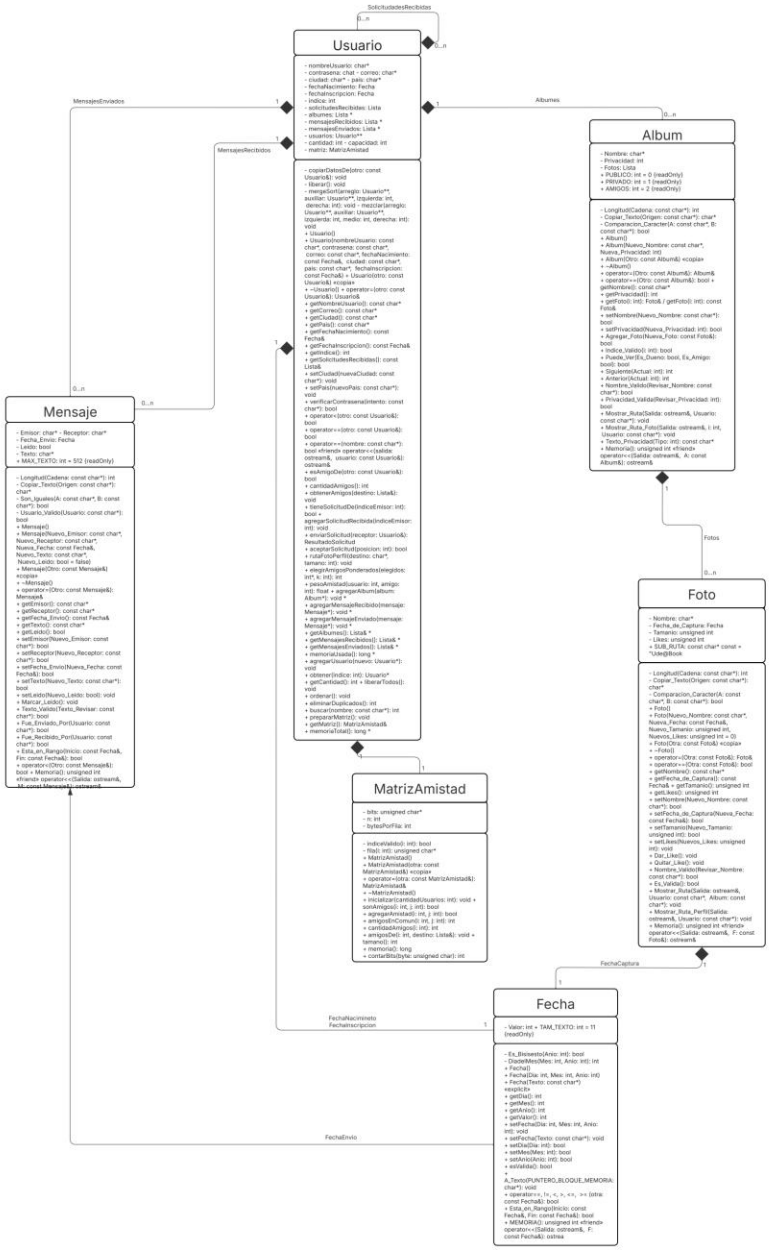
3.2 Actualización de los archivos

ARCHIVO	SE ACTUALIZA CUANDO
usuarios.txt	Nunca: en esta versión no hay registro ni edición de usuarios
amistades.txt	Se acepta una solicitud de amistad
solicitudes.txt	Se envía o se acepta una solicitud
mensajes.txt	Se envía un mensaje o se marcan mensajes como leídos
albumes.txt	Se da o se quita like a una foto

4. RESUMEN DE DIAGRAMA DE CLASES



4. DIAGRAMA DE CLASES COMPLETO



Marco 1

Clases de Apoyo

Recursos

- iteraciones: long

+ reiniciar(): void
+ sumar(n: long = 1): void
+ obtenerIteraciones(): long
+ memoriaTotal(): long *
+mostrarReporteFuncionalidad: const char*): void *

Persistencia

+ MAX_LINEA: int = 1024 {readOnly}

+ cargarUsuarios(ruta: const char*): bool
+ cargarAmistades(ruta: const char*): bool
+ guardarAmistades(ruta: const char*): bool
+ cargarSolicitudes(ruta: const char*): bool
+ guardarSolicitudes(ruta: const char*): bool
+ cargarMensajes(ruta: const char*): bool *
+ guardarMensajes(ruta: const char*): bool *
+ cargarAlbumes(ruta: const char*): bool *
+ guardarAlbumes(ruta: const char*): bool *
+ cargarTodo(carpeta: const char*): bool *
+ guardarTodo(carpeta: const char*): bool *
+ leerLinea(archivo: istream&, linea: char*, demasiadoLargo: bool&): bool
+ reemplazarArchivo(rutaTemporal: const char*, rutaFinal: const char*): bool

MenuAmistades

- INTENTOS_LOGIN: int = 3 {readOnly}
- FOTOS_A_MOSTRAR: int = 3 {readOnly}

+ iniciarSesion(): int
+ mostrarFotosPerfil(usuario: int): void
+ administrarAmistades(usuario: int): void
+ leerLinea(destino: char*, tamano: int): void
+ leerOpcion(): int
- verAmigos(usuario: int): void
- enviarSolicitud(usuario: int): void
- gestionarSolicitudes(usuario: int): void
- reportar(funcionalidad: const char*): void

Cadena

+ longitud(texto: const char*): int
+ copiar(texto: const char*): char*
+ comparar(a: const char*, b: const char*): int
+ iguales(a: const char*, b: const char*): bool
+ dividir(linea: char*, separador: char, campos: char**, maxCampos: int): int
+ quitarFinDeLinea(texto: char*): void
+ anexar(destino: char*, tamano: int, texto: const char*): void

Sugerencias

(sin atributos: se calculan con la matriz) *

+ calcular(usuario: int, k: int, sugeridos: int*, comunes: int*): int *
+ mostrar(usuario: int, k: int): void *

Lista

- datos: T*
- cantidad: int
- capacidad: int

- crecer(): void
- copiarDesde(otra: const Lista&): void
+ Lista()
+ Lista(otra: const Lista&) =copia=
+ operator=(otra: const Lista&): Lista&
+ ~Lista()
+ agregar(valor: const T&): void
+ eliminarEn(posicion: int): bool
+ buscar(valor: const T&): int
+ operator[](i: int): T& / operator[](i: int): const T&
+ tamano(): int
+ vacia(): bool
+ vaciar(): void
+ memoria(): long